Load data file

```
zip_path = "p2_data.zip"
```

```
# DO NOT EDIT THIS CELL

import zipfile

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall()

with open("./p2_data/readme.txt", "r") as f:
    print(f.read())
```

```
readme.txt:
-----------

Clean images are available in the folder "/p2_data/clean_images/"

There are 1000 clean images (10 classes * 100 images per class)

Image file path format is "/p2_data/clean_images/<class_name>_<index>.png"
     <class_name>: Name of the true class of the image
                   Can be among {airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck}
     <index>     : Index of the image ranging from 0 to 99

There is an empty folder "/p2_data/adv_images/". You can use it for saving your adversarial images.

The pretrained model is "/p2_data/cnn_model.pt"
```

Prepare the model and data

```
# DO NOT EDIT THIS CELL

import torch
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"

mean = (0.4914, 0.4822, 0.4465) # image mean
std  = (0.2470, 0.2435, 0.2616) # image standard deviation
class_names = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]
img_per_class = 100

# prepare input images
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize(mean, std)]) # specify transformations: covert to t
paths = [f"./p2_data/clean_images/{name}_{i}.png" for name in class_names for i in range(img_per_class)]
labels = [class_names.index(name) for name in class_names for _ in range(img_per_class)]
X = torch.stack([transform(Image.open(p).convert("RGB")) for p in paths], dim = 0).to(device) # convert to tensor and normalize
y_true = torch.tensor(labels).to(device) # convert to tensor

# load cnn model
model = torch.jit.load("./p2_data/cnn_model.pt", map_location = device)

# sanity check
y_pred = model(X)
matches = (y_true == y_pred)
print(f"Total images: {len(y_true)}  |  Correctly Classified: {sum(matches).item()}")

# show some images
print("Showing some sample images ...")
mean_t = torch.tensor(mean).view(3, 1, 1)
std_t  = torch.tensor(std).view(3, 1, 1)
fig, axes = plt.subplots(2, 5, figsize=(9, 3.5))
for i, idx in enumerate([59, 119, 283, 360, 427, 527, 682, 783, 806, 991]):
    img = (X[idx].cpu() * std_t + mean_t).clamp(0, 1)
    img = img.permute(1, 2, 0).numpy()
    ax = axes[i // 5, i % 5]
```

```
    ax.imshow(img)
    ax.set_title(f"{class_names[idx // img_per_class]} (label: {labels[idx]})", fontsize = 10)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

```
Total images: 1000  |  Correctly Classified: 1000
Showing some sample images ...
```



Adversarial Attack

```
# ============================================================
# BLACK-BOX, LABEL-ONLY, MINIMUM-L2 ATTACK  (HopSkipJump-style)
# Only uses the *hard label* returned by model(). No gradients,
# no scores, no architecture. All work is done in pixel space [0,1].
# ============================================================
import math, time, torch

torch.manual_seed(0)
N, D = len(X), 3 * 32 * 32
mean_t = torch.tensor(mean, device=device).view(1, 3, 1, 1)
std_t  = torch.tensor(std,  device=device).view(1, 3, 1, 1)

# ---------------- finalized attack parameters ----------------
SUBSET   = None
CHUNK    = 50
K_INIT   = 10
ITERS    = 40
N_RESTARTS = 2    # independent attack runs; the smallest-noise result is kept per image
N0       = 50     # gradient-estimation probes at iteration 1 (grows as sqrt(t))
N_MAX    = 800    # cap on probes per iteration
BS_STEPS = 12     # binary-search steps
THETA    = 1e-3   # probe radius factor
# -------------------------------------------------------------

# clean images in pixel space, exactly as stored in the PNG files
X01 = ((X * std_t + mean_t).clamp(0, 1) * 255).round() / 255

with torch.no_grad():
    y_clean = model(X)                       # model's prediction on the clean image
n_queries = torch.ones(N, device=device)     # per-image query counter (1 = the clean query)

@torch.no_grad()
def is_adv(x01, idx, quantize=False):
    """Query the black box. x01: (M,3,32,32) in [0,1]; idx: (M,) image ids.
    True  <=> predicted label is neither the true label nor the clean prediction."""
    x01 = x01.clamp(0, 1)
    if quantize:
        x01 = (x01 * 255).round() / 255      # what a saved 8-bit PNG would contain
    out = []
    for s in range(0, len(x01), 4096):
        out.append(model((x01[s:s + 4096] - mean_t) / std_t))
    p = torch.cat(out)
    n_queries.index_add_(0, idx, torch.ones(len(idx), device=device))
    return (p != y_true[idx]) & (p != y_clean[idx])

def bsearch(x0, xa, idx, steps=BS_STEPS, quantize=False):
```

```
        """xa must be adversarial. Returns the point on segment x0->xa closest to x0 that is adversarial."""
        lo = torch.zeros(len(x0), device=device)
        hi = torch.ones_like(lo)
        for _ in range(steps):
            mid = (lo + hi) / 2
            a = is_adv(x0 + mid.view(-1, 1, 1, 1) * (xa - x0), idx, quantize)
            hi = torch.where(a, mid, hi)
            lo = torch.where(a, lo, mid)
        out = x0 + hi.view(-1, 1, 1, 1) * (xa - x0)
        return ((out.clamp(0, 1) * 255).round() / 255) if quantize else out.clamp(0, 1)

    def l2(a, b):
        return (a - b).flatten(1).norm(dim=1)

    def init_points(idx, r=0):
        """Adversarial starting points: nearest other-class clean images, pulled towards the target by binary search."""
        M = len(idx)
        x0 = X01[idx]
        best = x0.clone()
        best_d = torch.full((M,), float("inf"), device=device)
        dist = torch.cdist(x0.view(M, -1), X01.view(N, -1))
        dist[y_true[None, :] == y_true[idx][:, None]] = float("inf")
        if r == 0:                                  # restart 0: the K nearest other-class images
            nn = dist.topk(K_INIT, largest=False).indices
        else:                                       # later restarts: a random K out of the 3K nearest
            pool = dist.topk(3 * K_INIT, largest=False).indices
            pick = torch.rand(M, 3 * K_INIT, device=device).argsort(1)[:, :K_INIT]
            nn = pool.gather(1, pick)
        for k in range(K_INIT):
            cand = X01[nn[:, k]]
            ok = is_adv(cand, idx)
            xb = bsearch(x0, cand, idx)
            dd = l2(xb, x0)
            upd = ok & (dd < best_d)
            best[upd], best_d[upd] = xb[upd], dd[upd]
        for _ in range(200):                        # fallback: random noise images (rarely needed)
            bad = torch.isinf(best_d).nonzero().squeeze(1)
            if len(bad) == 0:
                break
            cand = torch.rand(len(bad), 3, 32, 32, device=device)
            ok = is_adv(cand, idx[bad])
            xb = bsearch(x0[bad], cand, idx[bad])
            dd = l2(xb, x0[bad])
            g = bad[ok]
            best[g], best_d[g] = xb[ok], dd[ok]
        return best

    def hsja(idx, r=0):
        M = len(idx)
        x0 = X01[idx]
        xb = init_points(idx, r)
        for t in range(1, ITERS + 1):
            dist = l2(xb, x0)
            E = int(min(N0 * math.sqrt(t), N_MAX))
            delta = math.sqrt(D) * THETA * dist
            # --- estimate the boundary normal from label-only probes ---
            noise = torch.randn(M, E, 3, 32, 32, device=device)
            noise = noise / noise.flatten(2).norm(dim=2).view(M, E, 1, 1, 1)
            pert = xb[:, None] + delta.view(M, 1, 1, 1, 1) * noise
            idx_rep = idx[:, None].expand(M, E).reshape(-1)
            dec = is_adv(pert.view(-1, 3, 32, 32), idx_rep).view(M, E).float() * 2 - 1
            f = dec - dec.mean(1, keepdim=True)
            same = f.abs().sum(1) == 0
            f[same] = dec[same]
            grad = (f.view(M, E, 1, 1, 1) * noise).sum(1)
            grad = grad / (grad.flatten(1).norm(dim=1).view(M, 1, 1, 1) + 1e-12)
            # --- geometric step away from the boundary, then binary search back toward x0 ---
            eps = dist / math.sqrt(t)
            xn = xb.clone()
            pending = torch.arange(M, device=device)
            for _ in range(10):
                if len(pending) == 0:
                    break
                cand = (xb[pending] + eps[pending].view(-1, 1, 1, 1) * grad[pending]).clamp(0, 1)
                ok = is_adv(cand, idx[pending])
                xn[pending[ok]] = cand[ok]
                eps[pending[~ok]] /= 2
```

```
            pending = pending[~ok]
        new = bsearch(x0, xn, idx)
        better = l2(new, x0) < dist
        xb = torch.where(better.view(-1, 1, 1, 1), new, xb)
    return xb

def finalize(idx, xb):
    """Make sure the image is still adversarial after 8-bit rounding, then shrink the noise in the 8-bit domain."""
    x0 = X01[idx]
    scale = torch.ones(len(idx), device=device)
    xq = ((xb * 255).round()) / 255
    ok = is_adv(xq, idx)
    for _ in range(40):
        if ok.all():
            break
        j = (~ok).nonzero().squeeze(1)
        scale[j] *= 1.2                       # push further from the clean image until it survives 8-bit rounding
        xs = (x0[j] + scale[j].view(-1, 1, 1, 1) * (xb[j] - x0[j])).clamp(0, 1)
        xs = (xs * 255).round() / 255
        a = is_adv(xs, idx[j])
        xq[j[a]] = xs[a]
        ok[j[a]] = True
    out = bsearch(x0, xq, idx, steps=10, quantize=True)
    return out, ok

ids_all = torch.arange(N, device=device) if SUBSET is None else torch.arange(0, N, N // SUBSET, device=device)
X_adv01 = X01.clone()
best_l2 = torch.full((N,), float("inf"), device=device)   # best (smallest) valid noise found so far per image
t_start = time.time()
for r in range(N_RESTARTS):
    torch.manual_seed(r)
    for s in range(0, len(ids_all), CHUNK):
        idx = ids_all[s:s + CHUNK]
        xb = hsja(idx, r)
        xq, ok = finalize(idx, xb)
        d = l2(xq, X01[idx])
        upd = ok & (d < best_l2[idx])                      # keep only if valid AND smaller than before
        X_adv01[idx[upd]] = xq[upd]
        best_l2[idx[upd]] = d[upd]
        print(f"restart {r + 1}/{N_RESTARTS}  [{s + len(idx):4d}/{len(ids_all)}]  "
              f"best mean L2 so far = {best_l2[ids_all[:s + len(idx)]].mean():.4f}  "
              f"queries/img = {n_queries[ids_all[:s + len(idx)]].mean():.0f}  elapsed = {time.time() - t_start:.0f}s")
    print(f"=== after restart {r + 1}: mean L2 = {best_l2[ids_all].mean():.4f} ===")

failed_ids = ids_all[torch.isinf(best_l2[ids_all])].tolist()
with torch.no_grad():
    final_pred = model((X_adv01[ids_all] - mean_t) / std_t)
succ = ((final_pred != y_true[ids_all]) & (final_pred != y_clean[ids_all])).float().mean().item()
print(f"Attack success rate: {succ * 100:.2f}%   |   avg queries per image: {n_queries[ids_all].mean():.0f}")
print("Images that could not be attacked:", failed_ids)
```

```
restart 1/2  [  50/1000]  best mean L2 so far = 0.1812  queries/img = 9227  elapsed = 141s
restart 1/2  [ 100/1000]  best mean L2 so far = 0.1742  queries/img = 9228  elapsed = 296s
restart 1/2  [ 150/1000]  best mean L2 so far = 0.2125  queries/img = 9227  elapsed = 455s
restart 1/2  [ 200/1000]  best mean L2 so far = 0.2315  queries/img = 9227  elapsed = 612s
restart 1/2  [ 250/1000]  best mean L2 so far = 0.2202  queries/img = 9228  elapsed = 770s
restart 1/2  [ 300/1000]  best mean L2 so far = 0.2118  queries/img = 9228  elapsed = 928s
restart 1/2  [ 350/1000]  best mean L2 so far = 0.2069  queries/img = 9228  elapsed = 1086s
restart 1/2  [ 400/1000]  best mean L2 so far = 0.2041  queries/img = 9228  elapsed = 1243s
restart 1/2  [ 450/1000]  best mean L2 so far = 0.2035  queries/img = 9228  elapsed = 1400s
restart 1/2  [ 500/1000]  best mean L2 so far = 0.2017  queries/img = 9228  elapsed = 1558s
restart 1/2  [ 550/1000]  best mean L2 so far = 0.1984  queries/img = 9228  elapsed = 1716s
restart 1/2  [ 600/1000]  best mean L2 so far = 0.1952  queries/img = 9228  elapsed = 1874s
restart 1/2  [ 650/1000]  best mean L2 so far = 0.1970  queries/img = 9228  elapsed = 2031s
restart 1/2  [ 700/1000]  best mean L2 so far = 0.1997  queries/img = 9228  elapsed = 2189s
restart 1/2  [ 750/1000]  best mean L2 so far = 0.2018  queries/img = 9228  elapsed = 2347s
restart 1/2  [ 800/1000]  best mean L2 so far = 0.2044  queries/img = 9228  elapsed = 2505s
restart 1/2  [ 850/1000]  best mean L2 so far = 0.2066  queries/img = 9228  elapsed = 2663s
restart 1/2  [ 900/1000]  best mean L2 so far = 0.2100  queries/img = 9228  elapsed = 2822s
restart 1/2  [ 950/1000]  best mean L2 so far = 0.2152  queries/img = 9228  elapsed = 2979s
restart 1/2  [1000/1000]  best mean L2 so far = 0.2190  queries/img = 9228  elapsed = 3138s
=== after restart 1: mean L2 = 0.2190 ===
restart 2/2  [  50/1000]  best mean L2 so far = 0.1678  queries/img = 18453  elapsed = 3295s
restart 2/2  [ 100/1000]  best mean L2 so far = 0.1622  queries/img = 18454  elapsed = 3452s
restart 2/2  [ 150/1000]  best mean L2 so far = 0.2000  queries/img = 18454  elapsed = 3611s
restart 2/2  [ 200/1000]  best mean L2 so far = 0.2175  queries/img = 18453  elapsed = 3769s
restart 2/2  [ 250/1000]  best mean L2 so far = 0.2071  queries/img = 18454  elapsed = 3927s
restart 2/2  [ 300/1000]  best mean L2 so far = 0.1995  queries/img = 18455  elapsed = 4084s
```

```
restart 2/2  [ 350/1000]  best mean L2 so far = 0.1947  queries/img = 18455  elapsed = 4242s
restart 2/2  [ 400/1000]  best mean L2 so far = 0.1926  queries/img = 18455  elapsed = 4400s
restart 2/2  [ 450/1000]  best mean L2 so far = 0.1915  queries/img = 18455  elapsed = 4557s
restart 2/2  [ 500/1000]  best mean L2 so far = 0.1899  queries/img = 18455  elapsed = 4715s
restart 2/2  [ 550/1000]  best mean L2 so far = 0.1874  queries/img = 18455  elapsed = 4873s
restart 2/2  [ 600/1000]  best mean L2 so far = 0.1844  queries/img = 18455  elapsed = 5031s
restart 2/2  [ 650/1000]  best mean L2 so far = 0.1861  queries/img = 18455  elapsed = 5189s
restart 2/2  [ 700/1000]  best mean L2 so far = 0.1888  queries/img = 18455  elapsed = 5346s
restart 2/2  [ 750/1000]  best mean L2 so far = 0.1911  queries/img = 18455  elapsed = 5504s
restart 2/2  [ 800/1000]  best mean L2 so far = 0.1940  queries/img = 18455  elapsed = 5662s
restart 2/2  [ 850/1000]  best mean L2 so far = 0.1961  queries/img = 18455  elapsed = 5820s
restart 2/2  [ 900/1000]  best mean L2 so far = 0.1989  queries/img = 18455  elapsed = 5978s
restart 2/2  [ 950/1000]  best mean L2 so far = 0.2037  queries/img = 18455  elapsed = 6137s
restart 2/2  [1000/1000]  best mean L2 so far = 0.2073  queries/img = 18455  elapsed = 6295s
=== after restart 2: mean L2 = 0.2073 ===
Attack success rate: 100.00%   |   avg queries per image: 18455
Images that could not be attacked: []
```

```
import os
import numpy as np
from PIL import Image

ADV_DIR = "./adv_images"
os.makedirs(ADV_DIR, exist_ok=True)

# 8-bit RGB arrays (X_adv01 is already in pixel space [0,1], i.e. de-normalized)
arr = (X_adv01 * 255).round().clamp(0, 255).byte().permute(0, 2, 3, 1).cpu().numpy()
for i, p in enumerate(paths):
    name = os.path.basename(p)[:-4] + "_adv.png"          # cat_5.png -> cat_5_adv.png
    Image.fromarray(arr[i]).save(os.path.join(ADV_DIR, name))
print("Saved", len(paths), "images to", ADV_DIR)

# ---- verification: reload PNGs from disk exactly as the evaluator would ----
sel = ids_all.cpu().tolist()
X_re = torch.stack([transform(Image.open(os.path.join(ADV_DIR, os.path.basename(paths[i])[:-4] + "_adv.png")).convert("RGB"))
                    for i in sel]).to(device)
with torch.no_grad():
    y_adv = model(X_re)
print(f"Success rate (reloaded PNGs): {(y_adv != y_true[sel]).float().mean().item() * 100:.2f}%")
diff = (X_re * std_t + mean_t).clamp(0, 1) - X01[sel]
mse = diff.pow(2).flatten(1).mean(1)
print(f"Mean L2 (pixel space [0,1]): {diff.flatten(1).norm(dim=1).mean():.4f}")
print(f"Mean L-inf (0-255 scale):    {diff.abs().flatten(1).max(1).values.mean() * 255:.2f}")
print(f"Mean PSNR: {(10 * torch.log10(1 / mse.clamp_min(1e-12))).mean():.2f} dB")

# ---- robustness: the SAME model() call with different batch sizes (numerics can differ slightly with batch size/hardware) ----
with torch.no_grad():
    for bs in (len(X_re), 250, 32, 1):
        y_b = torch.cat([model(X_re[i:i + bs]) for i in range(0, len(X_re), bs)])
        print(f"batch size {bs:4d}: success {(y_b != y_true[sel]).float().mean().item() * 100:.2f}%")
print("Mean L2, normalized space:", (X_re - X[sel]).flatten(1).norm(dim=1).mean().item())

# ---- zip for download / GitHub (not a model call) ----
import shutil
zip_base = "/content/adv_images" if os.path.isdir("/content") else "./adv_images_zip"
shutil.make_archive(zip_base, "zip", ADV_DIR)
print("ZIP created:", zip_base + ".zip")
```

```
Saved 1000 images to ./adv_images
Success rate (reloaded PNGs): 100.00%
Mean L2 (pixel space [0,1]): 0.2073
Mean L-inf (0-255 scale):    4.95
Mean PSNR: 50.53 dB
batch size 1000: success 100.00%
batch size  250: success 100.00%
batch size   32: success 100.00%
batch size    1: success 100.00%
Mean L2, normalized space: 0.8331857323646545
ZIP created: /content/adv_images.zip
```